

File Upload / Media Storage System — Practical System Design Notes

Hinglish • S3 • Presigned URLs • Multipart Upload • CloudFront • Queue • Security

1. Problem Statement

User ko image, PDF, video, document ya other media upload karna hai. System ko upload efficiently handle karna, metadata store karna, files serve karna, access control lagana aur large files ko scale par process karna hai.

```
Client
  ↓
Backend
  ↓
Object Storage
  ↓
File
```

Metadata:
DB

Delivery:
CDN / Object Storage

2. The Basic Mistake

```
Client
  ↓
Node.js Server
  ↓
receive 500 MB file
  ↓
Node memory / disk
  ↓
S3
```

Problem:
Node becomes file-transfer bottleneck.

Large files ko unnecessarily Node.js ke through proxy karna bandwidth, CPU, memory, connections aur horizontal scaling ko unnecessarily expensive bana sakta hai.

3. Better Architecture — Presigned Upload

```
Client
  ↓
POST /upload/init
  ↓
Node.js
  ↓
Generate Presigned URL
  ↓
Client ██████████ → S3
                    ↓
                  Object
```

```
Client
  ↓
notify backend / metadata
  ↓
DB
```

Backend authorization/control rakhta hai, but actual bytes client directly object storage ko upload karta hai.

4. Why Presigned URL?

Presigned URL temporary permission deta hai jisse client directly S3 object ko upload/download kar sakta hai, without exposing long-lived cloud credentials to the browser.

Backend:
authenticate user
authorize upload
choose object key
generate short-lived URL
return URL

Client:
PUT file directly to S3

5. Upload Flow

1. User selects file
2. Client → POST /uploads/init
3. Backend authenticates
4. Validate type/size/policy
5. Generate object key
6. Generate presigned URL
7. Client uploads to S3
8. Backend records/updates metadata
9. Async processing starts

6. Object Key

```
uploads/  
  user-123/  
    2026/  
      09/  
        uuid-file.jpg
```

User-supplied filename ko directly storage key banana avoid karo. UUID/random identifier plus controlled prefix collision and naming issues reduce karta hai.

7. File Metadata DB

```
File  
-----  
id  
user_id  
object_key  
original_name  
content_type  
size  
status  
etag / checksum  
created_at  
updated_at  
deleted_at
```

```
Example status:  
PENDING  
UPLOADED  
PROCESSING  
READY  
FAILED  
DELETED
```

8. Why DB + S3?

S3/object storage actual bytes ke liye hai. DB searchable metadata, ownership, status, business relationships and permissions ke liye useful hai.

DB:
file_id → object_key → owner → status

S3:
object_key → actual bytes

9. Upload Status

```
PENDING  
↓  
UPLOADING  
↓  
UPLOADED  
↓  
PROCESSING  
↓  
READY
```

Failure:
PROCESSING → FAILED

Application ko file ko READY tab mark karna chahiye jab required validation/processing complete ho.

10. Large Files

Large files ke liye multipart upload use karo. File ko multiple parts mein divide karke parallel upload kiya ja sakta hai.

10 GB video

Part 1 ■■→ S3
Part 2 ■■→ S3
Part 3 ■■→ S3
...
Part N ■■→ S3

Complete Multipart Upload



Final Object

11. Multipart Upload Benefits

- Parallel upload.
- Failed part ko independently retry kar sakte ho.
- Large upload ko resume karna easier.
- Node server ko full file buffer nahi karni padti.

12. Multipart Failure

Part 1 ■
Part 2 ■
Part 3 ■
Part 4 ■

Retry Part 3 only

No need to restart entire upload.

Incomplete multipart uploads ko lifecycle/cleanup policy se remove karna important hai, warna abandoned parts storage cost create kar sakte hain.

13. Direct Upload vs Server Upload

Server Upload	Presigned Direct Upload
Client → Node → S3	Client → S3
Node handles bytes	Node handles authorization/control
More server bandwidth	Lower backend bandwidth
Simpler for small files	Better for scalable large/media uploads

14. Download Flow

Private file:

```
Client
↓
Backend authorization
↓
Presigned GET URL
↓
S3 / CDN
↓
File
```

Public/static files ke liye direct CDN/object URL possible hai. Private files ke liye authorization and signed access use cases common hain.

15. CloudFront

```
User
↓
CloudFront
↓ HIT
File

MISS
↓
S3
↓
CloudFront cache
↓
User
```

CDN edge locations users ke closer content serve kar sakti hain, reducing origin requests and improving latency for cacheable content.

16. Browser Cache vs CDN Cache

Browser cache user device par content cache karta hai. CDN cache edge locations par content cache karta hai. Dono independent layers hain aur Cache-Control/TTL policies carefully define karni hoti hain.

17. Cache Invalidation

New image version:
logo-v1.png
logo-v2.png

Versioned names
→ avoid frequent invalidation

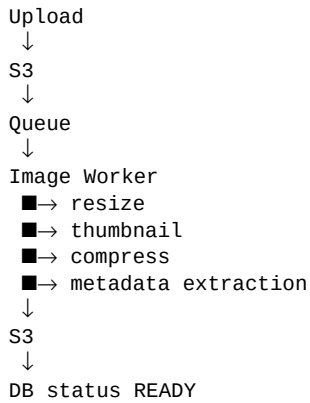
Or:
CDN invalidation
→ remove cached object

Static assets ke liye content/versioned filenames cache invalidation complexity reduce kar sakte hain.

18. Private Media

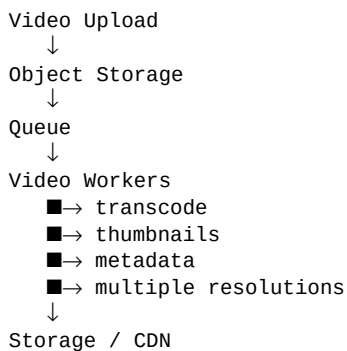
Private files ko public bucket/object ACL ke through expose karna avoid karo. Backend authorization ke baad short-lived signed URL ya controlled CDN access provide kar sakta hai.

19. Image Processing



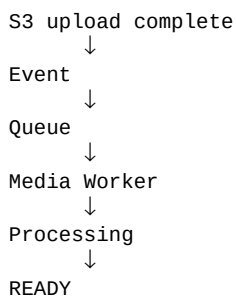
CPU-heavy image processing ko request path se remove karke asynchronous workers par run karna scalable approach hai.

20. Video Processing



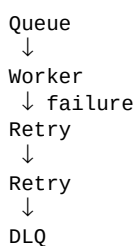
Video processing expensive ho sakti hai, isliye job queue, worker concurrency, retry and capacity planning important hain.

21. Queue



Queue spikes absorb karti hai aur upload API ko processing latency se decouple karti hai.

22. Retry + DLQ



Examples:
corrupt file

unsupported codec
temporary service failure

Permanent/poison jobs ko DLQ mein move karna useful hai. Retry policy error type ke according design karo.

23. Idempotent Processing

Worker crash ya duplicate event ke case mein same file ko multiple times process karne se inconsistent output nahi hona chahiye. File ID + processing version/job ID se idempotency key maintain ki ja sakti hai.

fileId = F123
processing = thumbnail-v2

Already READY?
→ skip/reuse

Not processed?
→ process

24. File Validation

- Maximum file size.
- Allowed MIME types.
- Extension checks.
- Magic-byte/content validation where needed.
- Image dimensions.
- Video duration/resolution.
- Filename/path normalization.
- Ownership and authorization.

25. MIME Type Is Not Enough

Client-provided Content-Type blindly trust nahi karna chahiye. File content/magic bytes and parser-based validation required ho sakti hai, especially security-sensitive uploads ke liye.

26. Virus / Malware Scanning

```
Upload
↓
Quarantine / PENDING
↓
Scanner
■→ clean → READY
■→ infected → REJECTED / QUARANTINED
```

Untrusted user uploads ko application users ke liye available karne se pehle security scanning required ho sakti hai, depending on product risk.

27. Upload Security

- Use short-lived presigned URLs.
- Keep bucket private where appropriate.
- Restrict object key/prefix to authorized user or tenant.
- Limit size and content types.
- Scan untrusted files.
- Avoid executing uploaded files.
- Protect against path/name manipulation.
- Apply least-privilege cloud IAM.
- Audit upload/download/delete operations.

28. Signed URL Expiry

Signed URL ka expiry short enough rakho ki leaked URL ka risk limited ho, but user upload/download complete kar sake. Exact TTL file size/network conditions aur security policy par depend karega.

29. Upload Progress

```
Client
↓
Multipart upload
↓
Part 1 20%
Part 2 40%
...
Part N 100%

Client UI:
■■■■■■■■■■ 100%
```

Browser upload progress client-side track kar sakta hai. Backend ko every byte ka synchronous update dena unnecessary ho sakta hai.

30. Resume Upload

Multipart upload mein successfully uploaded parts ke IDs/state maintain karke failed part retry/resume kiya ja sakta hai. Client session expiry aur abandoned upload cleanup bhi design karo.

31. Checksums / Integrity

Checksums upload integrity verify karne mein help karte hain. Storage/provider capabilities ke according checksum headers and multipart checksum strategy use ki ja sakti hai.

32. Duplicate File Upload

Same content multiple times upload ho sakta hai. Product requirement ke according duplicate detection content hash, user scope, object metadata ya simple unique business identifier se ki ja sakti hai.

```
file bytes
↓
SHA-256
↓
content hash

Already exists?
→ reuse/reference
or
→ store new object
```

Content deduplication optional complexity hai; storage savings ke saath security/privacy and ownership concerns bhi consider karo.

33. Storage Classes

Object storage systems often provide multiple storage classes with different cost/latency profiles. Frequently accessed media aur archival data ko same class mein rakhna zaroori nahi.

34. Lifecycle Rules

```
Object age
↓
30 days → cheaper storage
↓
180 days → archive
↓
delete after policy period
```

Lifecycle policies old files, incomplete multipart uploads and temporary objects ke cleanup mein useful hain.

35. Soft Delete

```
DB:
status = DELETED
deleted_at = timestamp
```

```
S3:
delete immediately
or
delayed cleanup
```

Business requirements ke according recovery window/retention policy maintain ki ja sakti hai.

36. Multi-Tenant Storage

```
tenant-a/
  files/...
```

```
tenant-b/
  files/...
```

```
Authorization:
user → tenant → object
```

Object key namespace, IAM policy and application authorization ko tenant isolation ke saath design karo.

37. Metadata Search

File metadata jaise owner, type, created date, tags and status DB mein searchable rakho. Large binary data ko relational metadata queries ke through fetch karne ki zaroorat nahi.

38. File Versioning

document.pdf

v1

v2

v3

DB:

file_id

version

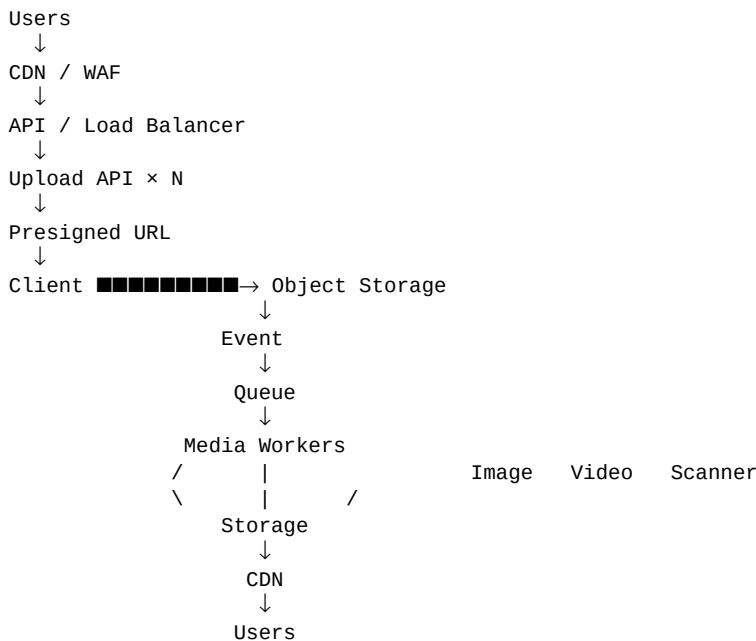
object_key

created_at

created_by

Document collaboration systems mein version history useful hoti hai. Object storage versioning bhi additional recovery layer provide kar sakti hai.

39. Large-Scale Architecture



Metadata:
DB

Fast state:
Redis (optional)

Monitoring:
Metrics + Logs + Traces

40. Upload Path vs Download Path

Upload path	Download path
Auth → presigned URL → direct storage	CDN/cache → object storage
Validation/security before access	Authorization for private content
Multipart for large files	Range requests/streaming where supported
Async processing after upload	Processed variants served to users

41. Range Requests / Video Streaming

Large media ko entire object download karne ke bajay byte ranges mein fetch karna useful ho sakta hai. CDN and object storage support ke according streaming architecture design ki ja sakti hai.

42. Hot Media

Popular image/video ko repeatedly origin se serve karna expensive ho sakta hai. CDN caching origin load reduce karti hai. Cache key and content versioning carefully design karo.

43. Origin Protection

Origin bucket ko public internet se directly expose karne ke bajay CDN-only/private access patterns use karna security posture improve kar sakta hai, depending on architecture.

44. Failure Scenarios

Failure	Handling
Presigned URL expires	Request a new upload URL; do not expose long-lived credentials
Multipart part fails	Retry failed part only
Worker crashes	Queue retry/idempotent processing
Scanner fails	Keep file non-READY; retry or quarantine
CDN miss	Fetch from origin/object storage
Redis down	Fallback or bypass depending on its role
DB down	Metadata operations may fail; existing cached/public media can behave differently
Orphaned S3 object	Lifecycle/reconciliation cleanup
Orphaned DB metadata	Reconciliation/repair jobs

45. Reconciliation

DB metadata aur object storage ke beech mismatch ho sakta hai. Periodic reconciliation jobs orphan objects, missing objects, incorrect status and failed processing detect kar sakte hain.

```
DB says READY
but object missing
↓
Mismatch
↓
repair / alert

Object exists
but DB has no reference
↓
orphan
↓
cleanup policy
```

46. Observability / Grafana

- Upload requests/sec
- Upload success/failure rate
- Presigned URL generation latency
- Multipart upload failures
- Average upload duration
- Object storage errors
- Queue depth/oldest job age
- Worker processing latency
- Scanner failure rate
- Thumbnail/transcoding latency
- CDN cache hit ratio
- Origin request rate
- Storage growth
- Download bandwidth
- 4xx/5xx rates

47. Cost Awareness

Storage cost ke alawa request count, data transfer, CDN transfer, processing compute and retained incomplete uploads cost create kar sakte hain. Large media systems mein cost metrics bhi monitor karna useful hai.

48. Interview Q&A;

Question	Short answer
Node se file directly S3 kyun nahi proxy karte?	Large files backend bandwidth, memory/connections aur scaling ko bottleneck kar sakti hain.
Presigned URL kya hai?	Temporary signed permission for direct object storage access.
Multipart upload kyun?	Large files, parallelism, part-level retry and resume.
Metadata DB mein kyun?	Ownership, status, search and business relationships.
S3 mein metadata kyun nahi?	Object metadata exists, but application queries/relationships ke liye DB more suitable.
CloudFront kyun?	Edge caching, lower latency and reduced origin load.
Private file kaise serve?	Authorize then signed URL/private CDN access.
Virus scanning?	Untrusted upload ko READY karne se pehle scan/quarantine.
Queue kyun?	Heavy image/video processing ko async and scalable banana.
Duplicate files?	Optional content hash/dedup strategy.
Orphan object?	Periodic reconciliation + lifecycle cleanup.
Large video?	Multipart upload + async transcoding + CDN.
Upload security?	Size/type validation, private storage, short-lived URLs, IAM, scanning.

49. Practice Scenarios

1. User uploads a 5 GB video. Design the complete flow.
2. User's network fails at 70% upload. How will you resume?
3. Why should Node.js not receive a 2 GB file directly?
4. Design private document download.
5. User uploads a malicious executable renamed as .jpg. What do you do?
6. Design image thumbnail generation.
7. One video becomes extremely popular. How do you protect S3/origin?
8. Design storage lifecycle for old videos.
9. DB says file READY but S3 object is missing. How do you recover?
10. S3 object exists but DB row is missing. How do you detect and clean it?
11. Design multi-tenant file storage.
12. Build metrics for upload and media processing.

50. Local Node.js Practice

Node.js + Express + AWS S3 + Redis + BullMQ + MySQL/PostgreSQL use karke project banao. Start with presigned upload, then multipart upload, metadata DB, private download, CloudFront, async thumbnail worker, scanning simulation, lifecycle/reconciliation and Prometheus/Grafana.

Phase 1:
POST /uploads/init
→ presigned URL

Phase 2:
Client → S3

Phase 3:
File metadata DB

Phase 4:
Private download
→ signed URL

Phase 5:
Multipart upload

Phase 6:
BullMQ
→ thumbnail worker

Phase 7:
Scanner simulation

Phase 8:
CloudFront

Phase 9:
Prometheus + Grafana

Phase 10:
reconciliation + cleanup

51. Final Cheat Sheet

FILE / MEDIA STORAGE

UPLOAD:
Client
↓
Backend auth + validation
↓
Presigned URL
↓
Client → S3
↓
Event / Queue
↓
Processing Workers
↓
READY

DOWNLOAD:
Client
↓
CDN / Signed URL
↓
S3
↓
Client

Must handle:
→ presigned URLs
→ multipart upload
→ resume/retry
→ metadata DB

- private access
- CloudFront
- image/video processing
- queue + workers
- virus scanning
- lifecycle cleanup
- reconciliation
- rate limits
- security
- observability

Golden interview line:

"I'll keep large file bytes out of the application servers by using short-lived presigned object-storage URLs. Metadata stays in the DB, media processing is asynchronous through a queue, private content uses authorized signed access, CloudFront serves hot media, and reconciliation/lifecycle policies clean up failures and abandoned objects."